MODULE: (2025) 6NTCM009W.2 Internet of Things

COURSEWORK ASSIGNMENT SPRING 2026

IoT prototype using Raspberry Pi Pico W and BME280 Sensor

Student: Hamza Hassan

Student ID: W20443810

Part A

Development Phase

Design and implementation of the IoT prototype

<u>Introduction</u>

This coursework documents the design, implementation, and evaluation of an IoT datalogging prototype built using a Raspberry Pi Pico W and a BME280 environmental sensor. The system captures real-time temperature readings and automatically transmits it to the cloud-based Google Sheets platform for storage, analysis, and visualisation. This allows for continuous monitoring, analysis, and visualisation of the environmental conditions across 3 test environments.

A1) Sensor Data (BME280):

The BME280 sensor was interfaced with the Raspberry Pi Pico W using the I2C communication protocol, allowing for reliable transfer of environmental data. The sensor then was used to measure different environmental parameters including temperature, humidity and pressure.

A MicroPython library was used to communicate with the sensor and retrieve readings. These values were then processed and displayed in the Thonny console. The temperature values were extracted and formatted to two decimal places for accuracy and consistency.

BME280 Sensor Reading Script ~

[bme_sensor_test.py]

```python
from machine import Pin, I2C
import bme280
from time import sleep

# I2C setup
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
bme = bme280.BME280(i2c=i2c)

# LED setup
led = Pin("LED", Pin.OUT)

led.on()

total_temp = 0
total_pressure = 0
total_humidity = 0

for i in range(5):
    temp = float(bme.values[0][:-1])
    pressure = float(bme.values[1][:-3])
    humidity = float(bme.values[2][:-1])

    print(f"Reading {i+1}: T={temp}C  H={humidity}%  P={pressure}hPa")

    total_temp += temp
    total_pressure += pressure
    total_humidity += humidity

    sleep(1)

avg_temp = total_temp / 5
avg_pressure = total_pressure / 5
avg_humidity = total_humidity / 5

print("\nAverages:")
print(f"Temperature: {avg_temp:.2f} C")
print(f"Humidity: {avg_humidity:.2f} %")
print(f"Pressure: {avg_pressure:.2f} hPa")

led.off()
```

Figure 1. MicroPython script used to read temperature, humidity, and pressure values from the BME280 sensor, calculate five-sample averages, and output the results to the Thonny console.

Shell Output Showing Sensor Readings ~

```
Shell ×
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Reading 1: T=17.5C  H=64.82%  P=1018.81hPa
  Reading 2: T=17.27C  H=64.8%  P=1018.87hPa
  Reading 3: T=17.26C  H=64.82%  P=1018.9hPa
  Reading 4: T=17.26C  H=64.82%  P=1018.79hPa
  Reading 5: T=17.25C  H=64.79%  P=1018.8hPa

  Averages:
  Temperature: 17.31 C
  Humidity: 64.81 %
  Pressure: 1018.83 hPa
```

Figure 2. Shell output generated by the BME280 sensor test script, showing five consecutive temperature, humidity, and pressure readings, followed by the calculated average values.

A2) – Web Server:

The Raspberry Pi Pico W was configured to act as a web server using its built-in Wi-Fi capability. The device connects to a local network and hosts a simple web page that displays real-time sensor readings.

When a user accesses the Pico's IP address through a browser, the server responds with updated environmental data. This allows remote monitoring of the sensor values without needing direct access to the device.

The web server will continuously update and refresh the displayed values, providing near real-time feedback. This behaviour is achieved through the inclusion of an HTML meta-refresh tag (<meta http-equiv="refresh" content="5">), which instructs the browser to reload the page every five seconds.

BME280 Web Server Script ~

[bme_web_server.py]

```python
import machine
from machine import Pin, I2C
from time import sleep
import network
import socket
import bme280

ssid = 'BT-P9ASNC'
password = 'mHQ7EunJUMJbVU'

# BME280 setup
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
bme = bme280.BME280(i2c=i2c)

def connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(ssid, password)

    while wlan.isconnected() == False:
        print("Waiting for connection ...")
        sleep(1)

    ip = wlan.ifconfig()[0]
    print(f'Connected on {ip}')
    return ip

def open_socket(ip):
    address = (ip, 80)
    connection = socket.socket()
    connection.bind(address)
    connection.listen(1)
    print(connection)
    return connection

def webpage(reading):
    html = f"""
        <!DOCTYPE html>
        <html>
        <head>
        <title>BME280 Sensor Readings</title>
        <meta http-equiv="refresh" content="5">
        </head>
        <body>
        <h2>BME280 Live Readings</h2>
```

```python
        <p>{reading}</p>
        </body>
        </html>
        """
    return str(html)

def serve(connection):
    while True:
        client = connection.accept()[0]
        request = client.recv(1024)
        request = str(request)

        temperature = bme.values[0]
        pressure = bme.values[1]
        humidity = bme.values[2]

        reading = f"Temperature: {temperature} | Pressure: {pressure} | Humidity: {humidity}"
        html = webpage(reading)

        client.send(html)
        client.close()

try:
    ip = connect()
    connection = open_socket(ip)
    serve(connection)
except KeyboardInterrupt:
    machine.reset()

```

Figure 3. MicroPython Script used to host a web server on the Raspberry Pi Pico W, retrieve live temperature, humidity, and pressure readings from the BME280 sensor, and serve them to any device on the local network. The script establishes a Wi-Fi connection, opens a socket on port 80, and generates an HTML page that automatically refreshes every five seconds to display updated environmental data.

Shell Output of Web Server Script ~

```
Shell
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Waiting for connection ...
  Waiting for connection ...
  Waiting for connection ...
  Waiting for connection ...
  Connected on 192.168.1.252
  <socket state=1 timeout=4294967295 incoming=0 off=0>
```

Figure 4. Shell output produced by the BME280 web server script, showing the Wi-Fi connection process, the assigned IP address, and confirmation that the socket is actively listening for incoming HTTP requests.

Live Sensor Readings Webpage (browser) ~

Figure 5. Webpage generated by the Raspberry Pi Pico W's built-in web server, displaying live temperature, humidity, and pressure readings retrieved from the BME280 sensor.

A3 – BME280 Data Logger:

A separate Google Sheet was created for Part A3 to demonstrate the functionality of the data logging system using 20 timestamped BME280 readings. This sheet includes temperature, pressure, and humidity values, which were later used to generate three graphs. A different dataset was used in Part B to support extended testing and analysis.

A cloud-based data logging system was implemented using Google Sheets and Google Apps Script. Initially, the Raspberry Pi Pico W was programmed to display 10 timestamped BME280 readings in the Thonny shell, including temperature, pressure, and humidity. This confirmed that the sensor data could be captured locally and viewed in real time.

The system was then extended so that the Pico W sent HTTP GET requests containing the timestamp, temperature, pressure, and humidity values to a deployed Google Apps Script web application. The script processed the incoming data and stored each reading in a new row within the spreadsheet. A counter was used to keep track of the row positions and ensure that the data was appended in order.

A total of 20 timestamped readings were successfully logged to the spreadsheet. These were used to generate three graphs: temperature vs time, pressure vs time, and humidity vs time. The results show that the system is capable of collecting, transmitting, storing, and visualising environmental data effectively.

This approach reflects a simple serverless IoT design, where the Pico W acts as the client device and Google Sheets acts as a lightweight cloud storage solution. While Google Sheets was sufficient for this prototype, the same concept could be expanded to more advanced

cloud platforms such as AWS IoT Core, AWS Lambda, or DynamoDB for larger-scale applications.

The use of HTTP requests provides a lightweight communication method between the device and the cloud, helping to keep the system efficient and scalable.

Micro Python logger script ~

[bme_data_logger_full.py]

```python
import machine
import time
from machine import Pin, I2C
import network
import urequests
import json
import gc
import bme280

wlan = network.WLAN(network.STA_IF)
board_led = machine.Pin("LED", machine.Pin.OUT)

ssid = "BT-P9ASNC"
password = "mHQ7EunJUMJbVU"

SCRIPT_URL = "https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec"
TIME_URL = "https://timeapi.io/api/Time/current/zone?timeZone=Europe/London"

# BME280 setup
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
bme = bme280.BME280(i2c=i2c)

def getTime():
    try:
        res = urequests.get(url=TIME_URL)
        timestamp = json.loads(res.text)["dateTime"]
        res.close()
        return timestamp
    except:
        print("Time API error")
        return "Time unavailable"

def connectWifi():
    wlan.active(True)
    if not wlan.isconnected():
        wlan.connect(ssid, password)
        while not wlan.isconnected() and wlan.status() >= 0:
            print("Waiting to connect...")
            time.sleep(1)
            board_led.off()
        board_led.on()
        print(wlan.ifconfig())
    else:
        print("Wifi already connected...")
```

```
        print(getTime())

def sendToSpreadsheet(time_value, temp, pressure, humidity):
    try:
        url = "{}?time={}&temp={}&pressure={}&humidity={}".format(
            SCRIPT_URL, time_value, temp, pressure, humidity
        )
        print(url)
        res = urequests.get(url=url)
        print(res.text)
        res.close()
        gc.collect()
    except:
        print("Error uploading data...")

connectWifi()
wlan.active(True)

for i in range(20):
    timestamp = "{}".format(getTime())

    temp = float(bme.values[0][:-1])
    pressure = float(bme.values[1][:-3])
    humidity = float(bme.values[2][:-1])

    temp_value = "{:.2f}".format(temp)
    pressure_value = "{:.2f}".format(pressure)
    humidity_value = "{:.2f}".format(humidity)

    print("Reading {}".format(i + 1))
    print("{} | Temperature: {}C | Pressure: {}hPa | Humidity: {}%".format(
        timestamp, temp_value, pressure_value, humidity_value
    ))

    sendToSpreadsheet(
        time_value=timestamp,
        temp=temp_value,
        pressure=pressure_value,
        humidity=humidity_value
    )

    time.sleep(5)

board led.off()
```

Figure 6. Micro Python A3 data logger script in Thonny

Shell Output ~

```
Shell ×
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Waiting to connect...
Waiting to connect...
Waiting to connect...
Waiting to connect...
('192.168.1.252', '255.255.255.0', '192.168.1.254', '192.168.1.254')
Reading 1
2026-03-19T00:34:30.1437951 | Temperature: 17.53C | Pressure: 1018.98hPa | Humidity: 64.20%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:34:30.1437951&temp=17.53&pressure=1018.98&humidity=6
4.20
ok
Reading 2
2026-03-19T00:34:40.7797597 | Temperature: 17.21C | Pressure: 1019.07hPa | Humidity: 64.25%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:34:40.7797597&temp=17.21&pressure=1019.07&humidity=6
4.25
ok
Reading 3
2026-03-19T00:34:52.6493897 | Temperature: 17.20C | Pressure: 1019.04hPa | Humidity: 64.26%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:34:52.6493897&temp=17.20&pressure=1019.04&humidity=6
4.26
ok
Reading 4
2026-03-19T00:35:03.8023968 | Temperature: 17.19C | Pressure: 1019.11hPa | Humidity: 64.34%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:35:03.8023968&temp=17.19&pressure=1019.11&humidity=6
4.34
ok
Reading 5
2026-03-19T00:35:14.1541416 | Temperature: 17.19C | Pressure: 1019.07hPa | Humidity: 64.34%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:35:14.1541416&temp=17.19&pressure=1019.07&humidity=6
4.34
ok
Reading 6
2026-03-19T00:35:24.696177 | Temperature: 17.19C | Pressure: 1019.08hPa | Humidity: 64.52%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:35:24.696177&temp=17.19&pressure=1019.08&humidity=64
.52
ok
Reading 7
2026-03-19T00:35:35.3379876 | Temperature: 17.19C | Pressure: 1019.06hPa | Humidity: 64.45%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:35:35.3379876&temp=17.19&pressure=1019.06&humidity=6
4.45
ok
Reading 8
2026-03-19T00:35:45.9945406 | Temperature: 17.18C | Pressure: 1019.09hPa | Humidity: 64.39%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:35:45.9945406&temp=17.18&pressure=1019.09&humidity=6
```

```
Reading 9
2026-03-19T00:35:57.1995139 | Temperature: 17.19C | Pressure: 1019.09hPa | Humidity: 64.45%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:35:57.1995139&temp=17.19&pressure=1019.09&humidity=6
4.45
ok
Reading 10
2026-03-19T00:36:07.8030333 | Temperature: 17.18C | Pressure: 1019.09hPa | Humidity: 64.36%
https://script.google.com/macros/s/AKfycbxYidwp0h3sdz2YM5mq73sjBw-B-FvxG0Cv7dy5O3E2IsCwBdxGiPEvdv7TIY87yIkUkA/exec?time=2026-03-19T00:36:07.8030333&temp=17.18&pressure=1019.09&humidity=6
4.36
ok
```

Figure 7. Shell output showing 10 timestamped BME280 readings (temperature, pressure, humidity).

Google Apps Script for Cloud Data Logging ~

```google apps script
SHEET_NAME = "A3_BME_Logger"
COUNTER_CELL = "E2"

function doGet(e) {
  var spreadSheet = SpreadsheetApp.getActiveSpreadsheet();
  var SHEET = spreadSheet.getSheetByName(SHEET_NAME);

  if (!SHEET) {
    return ContentService.createTextOutput("Sheet not found");
  }

  lastLog = SHEET.getRange(COUNTER_CELL).getValue();
  if (!lastLog) {
    lastLog = 1
  }

  lastLog = lastLog + 1;

  SHEET.getRange(COUNTER_CELL).setValue(lastLog);
  SHEET.getRange("A" + lastLog).setValue(e.parameter.time);
  SHEET.getRange("B" + lastLog).setValue(e.parameter.temp);
  SHEET.getRange("C" + lastLog).setValue(e.parameter.pressure);
  SHEET.getRange("D" + lastLog).setValue(e.parameter.humidity);

  return ContentService.createTextOutput("ok");
}
```

Figure 8. Google Apps Script used to receive and store BME280 data in Google Sheets

Google Sheet BME280 data logger ~

	A	B	C	D	E
1	Timestamp	Temperature (°C)	Pressure	Humidity	Counter
2	2026-03-19 0:34	17.53	1018.98	64.2	21
3	2026-03-19 0:34	17.21	1019.07	64.25	
4	2026-03-19 0:34	17.2	1019.04	64.26	
5	2026-03-19 0:35	17.19	1019.11	64.34	
6	2026-03-19 0:35	17.19	1019.07	64.34	
7	2026-03-19 0:35	17.19	1019.08	64.52	
8	2026-03-19 0:35	17.19	1019.06	64.45	
9	2026-03-19 0:35	17.18	1019.09	64.39	
10	2026-03-19 0:35	17.19	1019.09	64.45	
11	2026-03-19 0:36	17.18	1019.09	64.36	
12	2026-03-19 0:36	17.18	1019.08	64.37	
13	2026-03-19 0:36	17.17	1019.03	64.36	
14	2026-03-19 0:36	17.17	1019.07	64.46	
15	2026-03-19 0:36	17.17	1019.01	64.37	
16	2026-03-19 0:37	17.16	1019.1	64.3	
17	2026-03-19 0:37	17.17	1019.05	64.31	
18	2026-03-19 0:37	17.18	1019.12	64.46	
19	2026-03-19 0:37	17.17	1019.07	64.5	
20	2026-03-19 0:37	17.17	1019.11	64.38	
21	2026-03-19 0:38	17.17	1019.08	64.38	

Figure 9. Google Sheet showing 20 timestamped BME280 readings

Temperature vs Time ~

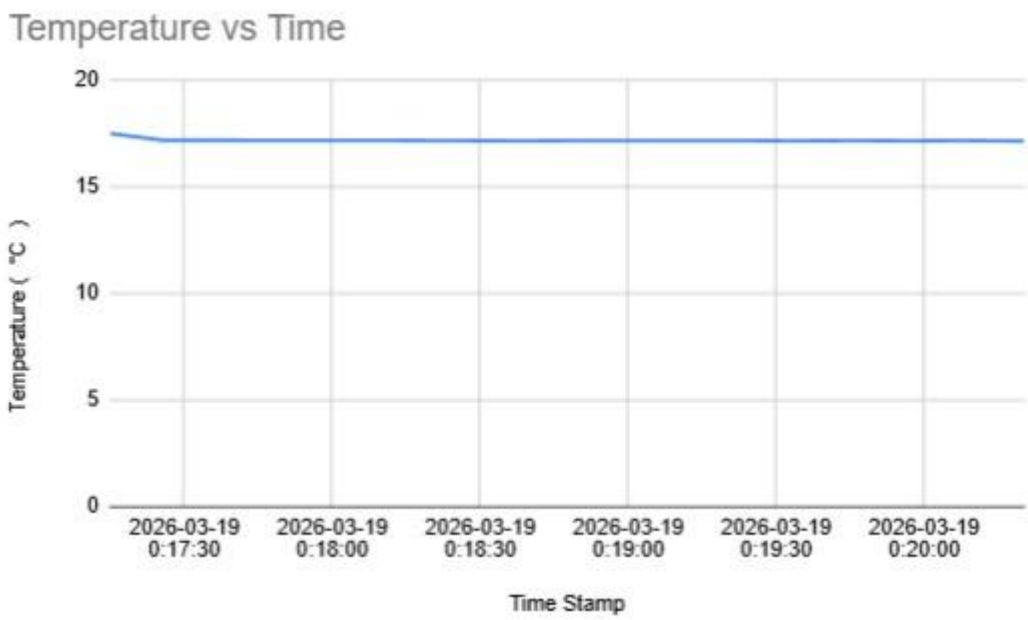

Figure 10. Temperature vs Time graph

Pressure vs Time ~

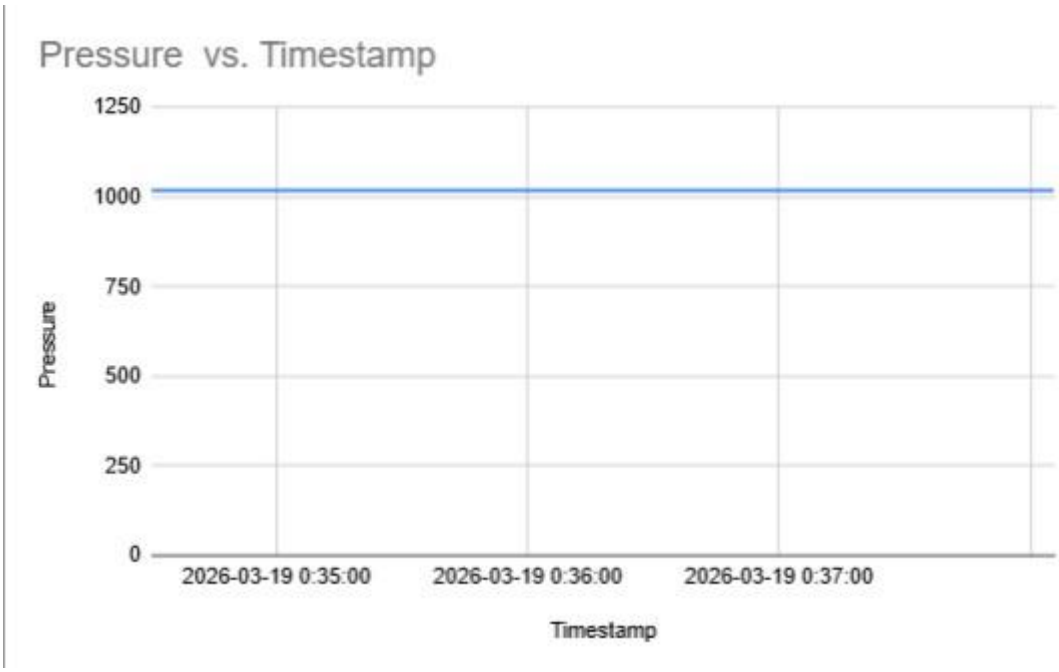

Figure 11. Pressure vs Time graph

Humidity vs Time ~

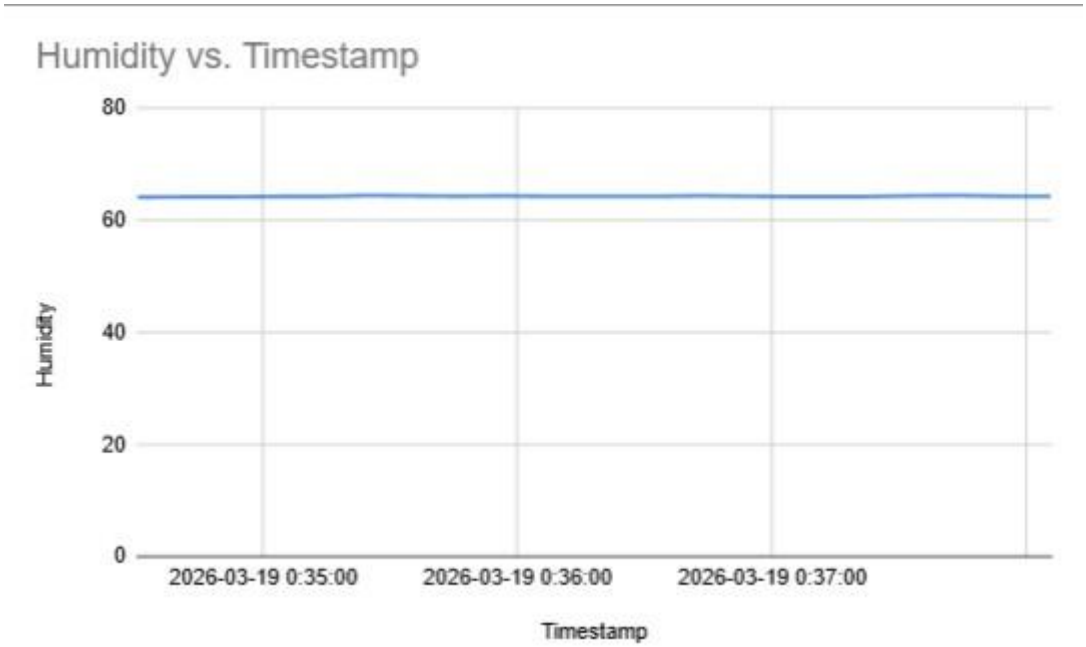

Figure 12. Humidity vs Time graph

Part B Testing Phase

The testing phase evaluates the performance of the IoT system using temperature data collected across multiple environments to assess accuracy, consistency, and real-world reliability.

B1) – Data Collection and Testing Environments:

A total of 182 temperature readings were collected across three distinct indoor environments to evaluate the performance of the Raspberry Pi Pico W and BME280 sensor under varying conditions.

The locations used were a bedroom, a university environment, and a living room. Approximately 60 readings were collected in each location, resulting in 61 readings in the bedroom, 60 readings in the university environment, and 61 readings in the living room.

The readings were logged automatically to the same Google Sheet using the cloudbased data logger developed in Part A. A location label was also added to distinguish between the three environments and improve clarity of the analysis

Testing Environment	**Reading Range (Values)**	**Total Values Collected**
My Room (At Home)	2 to 62	61
University of Westminster (Harrow Campus)	63 to 122	60
My Living Room (At Home)	123 to 183	61

B2) – Graph & Results:

Temperature vs Location ~

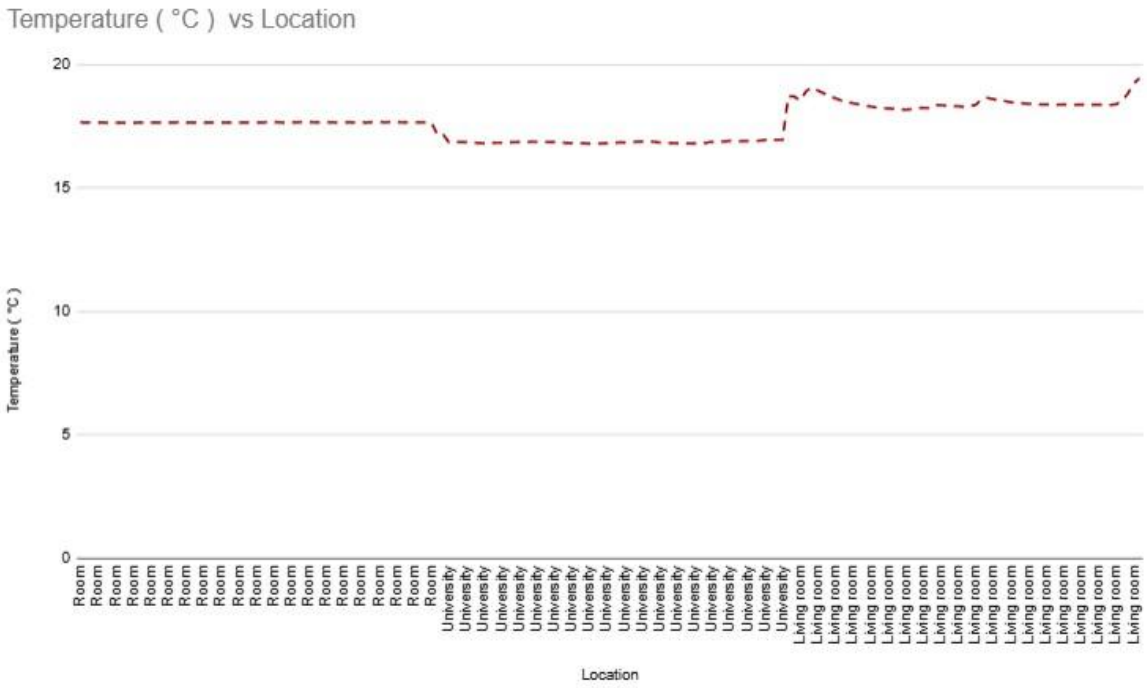

Figure 13. shows the variation in temperature readings across the three test environments. The bedroom readings were relatively stable and remained around 17.6 °C to 17.7 °C. The university readings were slightly lower, generally ranging from around 16.8 °C to 17.0 °C, showing the coolest environment among the three. The living room readings were the highest, increasing to approximately 18.5 °C to 19.0 °C.

Temperature vs Reading Progression ~

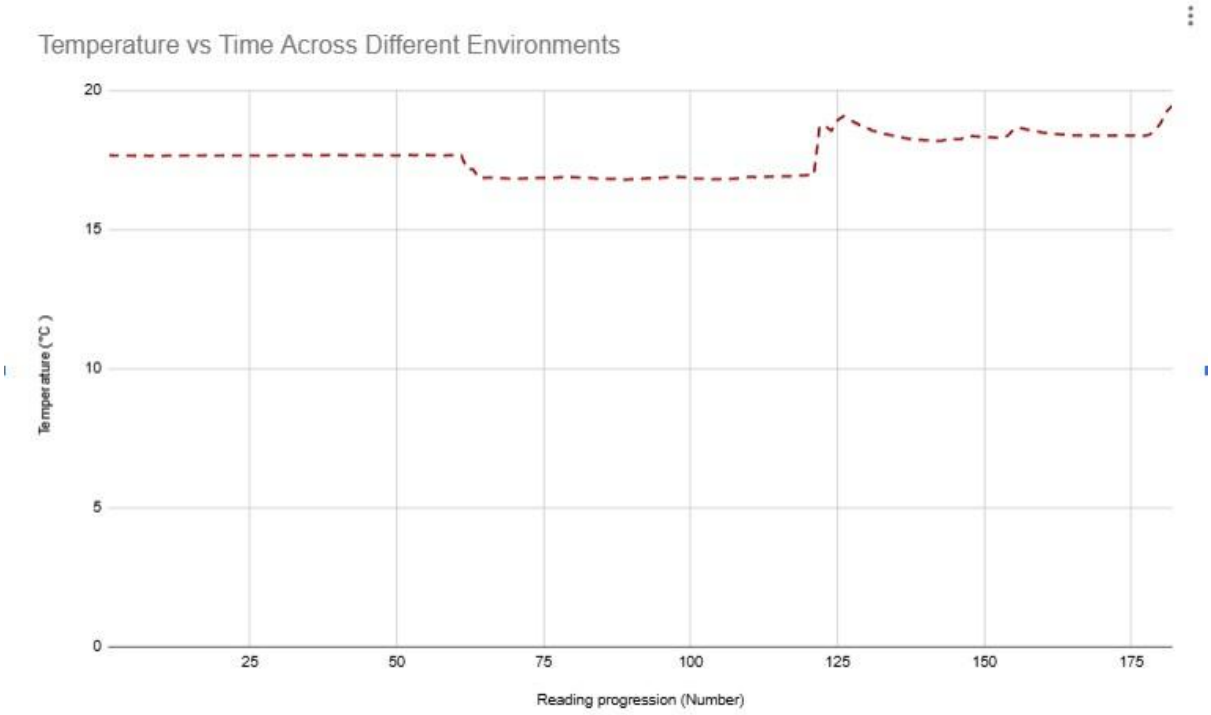

Figure 14. Temperature variation over time across different environments, plotted using the recorded sensor readings. The x-axis represents the reading progression (number of samples taken), while the y-axis represents the measured temperature in °C.

Both graphs demonstrate that the BME280 sensor was able to detect clear differences between the environments. This confirms that the prototype can successfully collect and visualise temperature changes over time. The use of reading progression on the X – axis also makes the differences between the three environments easier to interpret.

With my room at home spanning rows 2 to 62, the University of Westminster (Harrow Campus) covering rows 63 to 122, and my living room at home ranging from rows 123 to 183.

The observed differences in temperature across the environments can be explained by environmental conditions such as airflow, insulation, and human activity. The university environment likely experienced lower temperatures due to increased ventilation and larger open spaces, while the living room recorded higher temperatures due to enclosed space and possible heat sources. This demonstrates that the sensor is responsive to real-world environmental changes.

B3) – Maximum / Minimum / Average:

The maximum temperature recorded was 19.46 °C at *18/03/2026* | 20:10:31 The minimum temperature recorded was 16.80 °C at *17/03/2026* | 22:15:15.
The overall average temperature across all environmental readings was 17.74 °C.

The maximum temperature was observed in the living room environment, which produced the warmest readings overall. The minimum temperature occurred in the university environment, which produced the coolest readings. The average value provides an overall summary of the environmental conditions measured throughout the testing process.

Temperature Summary by Environment

Bedroom:

The maximum temperature recorded was 17.68 °C at *17/03/2026* | *20:13:18*
The minimum temperature recorded was 17.65 °C at 17/03/2026 | *20:08:36* The overall average temperature across all environmental readings was 17.66 °C.

University:

The maximum temperature recorded was 16.96 °C at *17/03/2026* | *22:20:21* The minimum temperature recorded was 16.80 °C at *17/03/2026* | 22:15:15.
The overall average temperature across all environmental readings was 16.86 °C.

Living room:

The maximum temperature recorded was 19.46 °C at 18/03/2026 | 20:10:31
The minimum temperature recorded was 18.18 °C at 18/03/2026 | *20:04:18* The overall average temperature across all environmental readings was 18.55 °C.

B4) – Connection Loss:

During testing, potential connection issues were considered when sending HTTPS requests to the Google Apps Script and time API. In IoT systems, SSL-related connection errors can occur when the Pico W loses its secure connection during repeated requests.

To address this, error handling was implemented using try-except blocks. This approach prevents the program from crashing and allows the system to continue operating or retry the request if a failure occurs. A practical improvement would be to catch connection-related exceptions, print an error message, and attempt to reconnect to the Wi-Fi network or resend the failed request.

This improves the robustness of the IoT prototype by ensuring that temporary communication failures do not completely stop the data logging process. In real-world deployments, such handling is essential for maintaining reliability and reducing data loss.

For example, try-except blocks can be implemented in the getTime() and data upload functions to handle network-related failures more effectively.

```
def getTime():
    res = urequests.get(url=TIME_URL)
    timestamp = json.loads(res.text)["dateTime"]
    res.close()
    return timestamp
```

Figure 15. The getTime() function used to retrieve the current timestamp from an online time API.

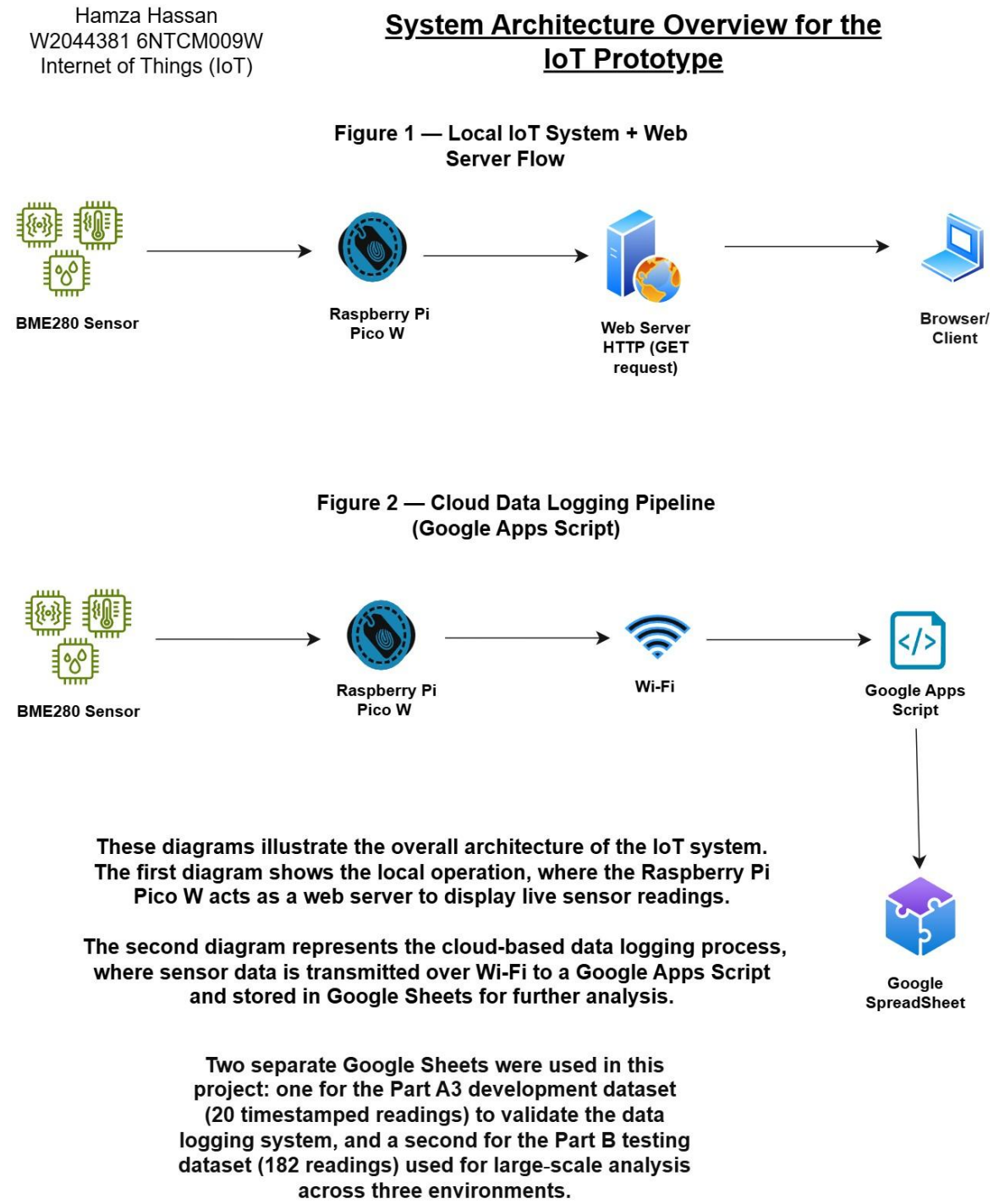

Figure 16. IoT System Architecture Overview

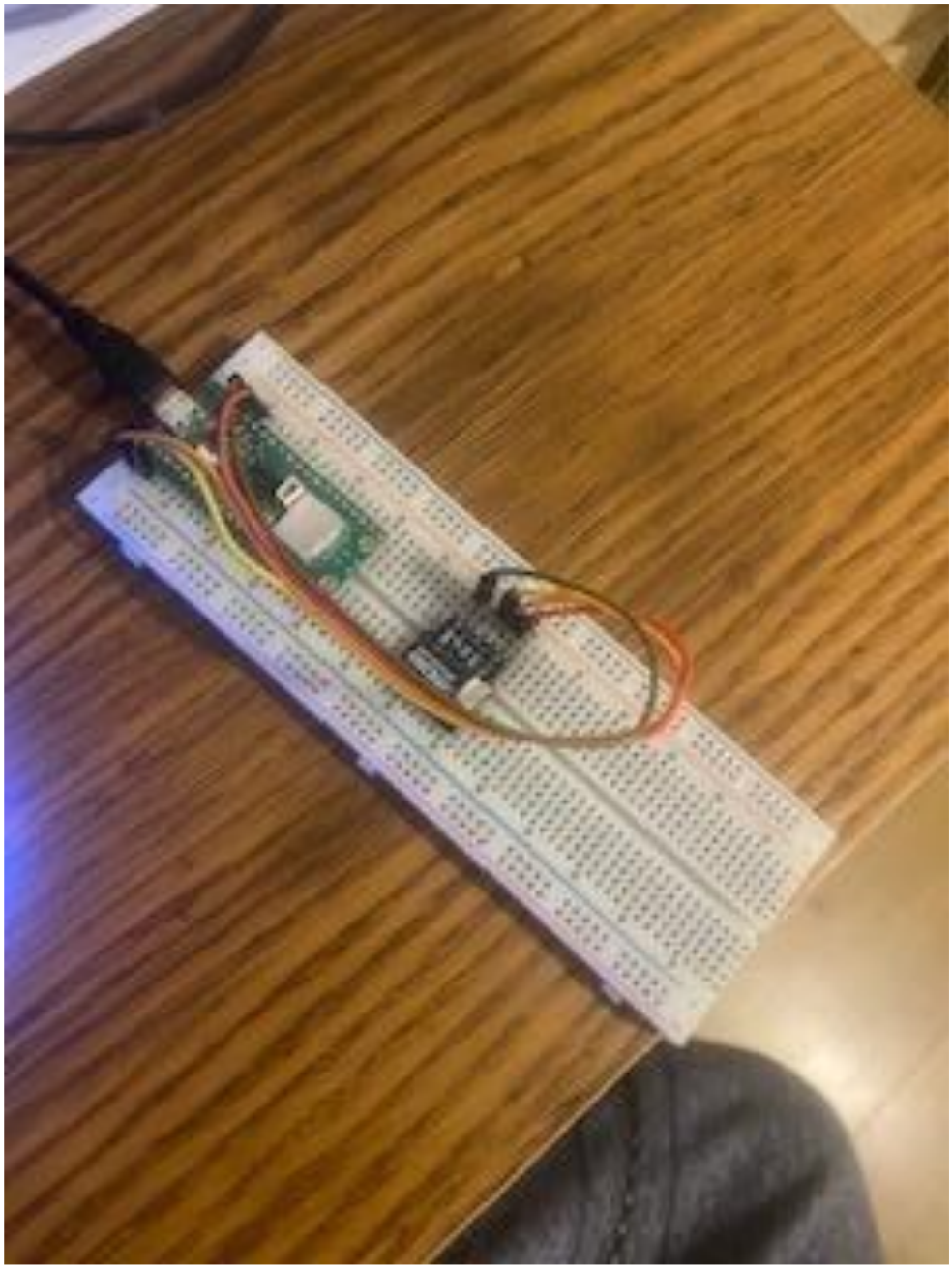

Figure 17. Deployed Raspberry Pi Pico W connected to the BME280 sensor using I2C wiring

Conclusion

This project successfully demonstrated the development of an IoT prototype using the Raspberry Pi Pico W and BME280 sensor, capable of collecting, transmitting, and visualising environmental data in real time. The system was able to integrate local processing, web-based monitoring, and cloud-based storage into a single working solution. Separating the A3 data logger from the Part B testing setup allowed for a clear distinction between system validation and large-scale data analysis. The results obtained across different environments showed consistent and realistic temperature variations, confirming both the accuracy of the sensor and the reliability of the data logging system.

A limitation of this prototype is the potential for minor sensor drift and occasional Wi-Fi-related transmission delays, which may introduce small variations in the recorded data.

Overall, this work highlights how low-cost hardware and simple cloud services can be effectively combined to build scalable IoT solutions, while also demonstrating practical considerations such as error handling and system robustness.

Google Sheets Data Access:

A3 – Development Dataset (20 timestamped readings: temperature, pressure, humidity):
https://docs.google.com/spreadsheets/d/1_w4BOo52XXRRu4r15p9Vx5EQout6QWocLsEpQt9LAg

Part B – Testing Dataset (extended temperature readings across environments):
https://docs.google.com/spreadsheets/d/17RymEau41x8c0HXado8xhYjZqqqz7B3_lNq15FICP48